

AI ASSIGNMENT 2

23L-2506

Question 1

```
import random
```

```
# =====  
# PART (a): Core Implementations  
# =====
```

```
def get_neighbours(state, n):  
    neighbours = []  
    if state > 1:  
        neighbours.append(state - 1)  
    if state < n:  
        neighbours.append(state + 1)  
    return neighbours
```

```
def first_choice_hc(landscape, start):  
    n = len(landscape)  
    current = start  
    path = [current]  
    while True:  
        neighbours = get_neighbours(current, n)  
        moved = False  
        for neighbour in neighbours:  
            if landscape[neighbour - 1] > landscape[current - 1]:  
                current = neighbour  
                path.append(current)  
                moved = True  
                break  
        if not moved:  
            break  
    return path, current
```

```
def stochastic_hc(landscape, start):  
    n = len(landscape)  
    current = start
```

```

path = [current]
while True:
    neighbours = get_neighbours(current, n)
    uphill = [nb for nb in neighbours
               if landscape[nb - 1] > landscape[current - 1]]
    if not uphill:
        break
    current = random.choice(uphill)
    path.append(current)
return path, current

```

```

# =====
# PART (b): Main block
# =====

```

```

def main():
    landscape = [4, 9, 6, 11, 8, 15, 10, 7, 13, 5, 16, 12]
    n = len(landscape)

    print("=" * 75)
    print("PART (a)/(b) -- Original Landscape")
    print("{:<6} {:<6}".format("State", "f(s)"))
    for i in range(n):
        print(" {:<4} {:<4}".format(i + 1, landscape[i]))
    print()

    random.seed(42)

    header = "{:<6} {:<20} {:<35} {:<10} {:<6}".format(
        "Start", "Algorithm", "Path", "Terminal", "Steps")
    print(header)
    print("-" * 77)

    fc_global = 0
    st_global = 0
    global_max_state = landscape.index(max(landscape)) + 1

    fc_results = {}
    st_results = {}

    for start in range(1, n + 1):
        path_fc, term_fc = first_choice_hc(landscape, start)
        steps_fc = len(path_fc) - 1

```

```

path_str = " -> ".join(str(s) for s in path_fc)
print("{:<6} {:<20} {:<35} {:<10} {:<6}".format(
    start, "First-Choice HC", path_str, term_fc, steps_fc))
fc_results[start] = (path_fc, term_fc)
if term_fc == global_max_state:
    fc_global += 1

path_st, term_st = stochastic_hc(landscape, start)
steps_st = len(path_st) - 1
path_str = " -> ".join(str(s) for s in path_st)
print("{:<6} {:<20} {:<35} {:<10} {:<6}".format(
    start, "Stochastic HC", path_str, term_st, steps_st))
st_results[start] = (path_st, term_st)
if term_st == global_max_state:
    st_global += 1

print()

print("=" * 75)
print("SUMMARY TABLE: Starting states reaching global maximum (state 11, f=16)")
print("{:<22} {:<25} {}".format("Algorithm", "Count reaching state 11", "Out of 12"))
print("-" * 55)
print("{:<22} {:<25} {}".format("First-Choice HC", fc_global, 12))
print("{:<22} {:<25} {}".format("Stochastic HC", st_global, 12))

print()
print("=" * 75)
print("DIVERGENCE: States where FC and Stochastic HC reach DIFFERENT terminals")
print("{:<6} {:<14} {:<14} {}".format("Start", "FC Terminal", "ST Terminal", "Reason"))
print("-" * 75)
diverged = False
for start in range(1, n + 1):
    _, t_fc = fc_results[start]
    _, t_st = st_results[start]
    if t_fc != t_st:
        diverged = True
        print("{:<6} {:<14} {:<14} FC chose left first (f={}); ST chose right (f={})".format(
            start, t_fc, t_st, landscape[t_fc - 1], landscape[t_st - 1]))
if not diverged:
    print(" No divergence observed in this run (stochastic seed=42).")

print()
print("=" * 75)
print("50-RUN EXPERIMENT: Stochastic HC from start = 4 (no fixed seed)")

```

```

print("-" * 45)
count_global = 0
count_local = 0
results_50 = {}
for _ in range(50):
    _, term = stochastic_hc(landscape, 4)
    results_50[term] = results_50.get(term, 0) + 1
    if term == global_max_state:
        count_global += 1
    else:
        count_local += 1

print("{:<16} {:<8} {}".format("Terminal State", "Count", "Percentage"))
print("-" * 35)
for state, cnt in sorted(results_50.items()):
    marker = "<< GLOBAL MAX" if state == global_max_state else "<< local max"
    print(" State {:<10} {:<8} {:.1f}%{}".format(state, cnt, 100 * cnt / 50, marker))
print()
print("Reached global max (state 11): {} / 50 = {:.1f}%".format(
    count_global, 100 * count_global / 50))
print("Stuck at local max: {} / 50 = {:.1f}%".format(
    count_local, 100 * count_local / 50))

```

```

# =====
# PART (c): Plateau Handling
# =====

```

```

def first_choice_hc_plateau(landscape, start, sideways=False, max_sideways=10):
    n = len(landscape)
    current = start
    path = [current]
    sideways_count = 0
    warning_printed = False

    while True:
        neighbours = get_neighbours(current, n)
        moved = False
        equal_exists = False

        for neighbour in neighbours:
            nb_val = landscape[neighbour - 1]
            cur_val = landscape[current - 1]
            if nb_val > cur_val:

```

```

        current = neighbour
        path.append(current)
        moved = True
        break
    elif nb_val == cur_val:
        equal_exists = True

if not moved:
    if equal_exists and not warning_printed:
        print(" [PLATEAU WARNING] FC: state {} (f={}) "
              "-- no strictly better neighbour but equal neighbour exists.".format(
                  current, landscape[current - 1]))
        warning_printed = True

if sideways and equal_exists and sideways_count < max_sideways:
    for neighbour in neighbours:
        if landscape[neighbour - 1] == landscape[current - 1]:
            current = neighbour
            path.append(current)
            sideways_count += 1
            moved = True
            warning_printed = False
            break

if not moved:
    break

return path, current

```

```

def stochastic_hc_plateau(landscape, start, sideways=False, max_sideways=10):
    n = len(landscape)
    current = start
    path = [current]
    sideways_count = 0
    warning_printed = False

    while True:
        neighbours = get_neighbours(current, n)
        cur_val = landscape[current - 1]
        uphill = [nb for nb in neighbours if landscape[nb - 1] > cur_val]
        equal = [nb for nb in neighbours if landscape[nb - 1] == cur_val]

        if uphill:

```

```

    current = random.choice(uphill)
    path.append(current)
    warning_printed = False
else:
    if equal and not warning_printed:
        print(" [PLATEAU WARNING] ST: state {} (f={}) "
              "-- no strictly better neighbour but equal neighbour exists.".format(
                  current, landscape[current - 1]))
        warning_printed = True

    if sideways and equal and sideways_count < max_sideways:
        current = random.choice(equal)
        path.append(current)
        sideways_count += 1
        warning_printed = False
    else:
        break

return path, current

```

```

def run_plateau_experiments():
    landscape_p = [4, 9, 6, 11, 15, 15, 15, 7, 13, 5, 16, 12]
    n = len(landscape_p)
    global_max_state = landscape_p.index(max(landscape_p)) + 1

    random.seed(42)

    print()
    print("=" * 75)
    print("PART (c) -- PLATEAU LANDSCAPE: f(5)=f(6)=f(7)=15")
    print("{:<6} {:<6}".format("State", "f(s)"))
    for i in range(n):
        print(" {:<4} {:<4}".format(i + 1, landscape_p[i]))

    print()
    print("-" * 75)
    print("RUN WITHOUT SIDEWAYS MOVES (plateau detection only)")
    print("-" * 75)

    fc_stuck = 0
    st_stuck = 0
    fc_global_p = 0
    st_global_p = 0

```

```

header = "{:<6} {:<20} {:<40} {:<10} {}".format(
    "Start", "Algorithm", "Path", "Terminal", "Stuck?")
print(header)
print("-" * 80)

for start in range(1, n + 1):
    path_fc, term_fc = first_choice_hc_plateau(landscape_p, start, sideways=False)
    stuck_fc = (term_fc != global_max_state)
    if stuck_fc:
        fc_stuck += 1
    else:
        fc_global_p += 1
    path_str = " -> ".join(str(s) for s in path_fc)
    print("{:<6} {:<20} {:<40} {:<10} {}".format(
        start, "FC (no sideways)", path_str, term_fc, "YES" if stuck_fc else "no"))

    path_st, term_st = stochastic_hc_plateau(landscape_p, start, sideways=False)
    stuck_st = (term_st != global_max_state)
    if stuck_st:
        st_stuck += 1
    else:
        st_global_p += 1
    path_str = " -> ".join(str(s) for s in path_st)
    print("{:<6} {:<20} {:<40} {:<10} {}".format(
        start, "ST (no sideways)", path_str, term_st, "YES" if stuck_st else "no"))
    print()

print("FC stuck on plateau: {}/12 (reached global max: {}/12)".format(fc_stuck, fc_global_p))
print("ST stuck on plateau: {}/12 (reached global max: {}/12)".format(st_stuck, st_global_p))

print()
print("-" * 75)
print("RUN WITH SIDEWAYS MOVES (cap = 10 per run)")
print("-" * 75)

fc_stuck_sw = 0
st_stuck_sw = 0
fc_global_sw = 0
st_global_sw = 0

random.seed(42)

print(header)

```

```

print("-" * 80)

for start in range(1, n + 1):
    path_fc, term_fc = first_choice_hc_plateau(
        landscape_p, start, sideways=True, max_sideways=10)
    stuck_fc = (term_fc != global_max_state)
    if stuck_fc:
        fc_stuck_sw += 1
    else:
        fc_global_sw += 1
    path_str = " -> ".join(str(s) for s in path_fc)
    print("{:<6} {:<20} {:<40} {:<10} {}".format(
        start, "FC (sideways<=10)", path_str, term_fc, "YES" if stuck_fc else "no"))

    path_st, term_st = stochastic_hc_plateau(
        landscape_p, start, sideways=True, max_sideways=10)
    stuck_st = (term_st != global_max_state)
    if stuck_st:
        st_stuck_sw += 1
    else:
        st_global_sw += 1
    path_str = " -> ".join(str(s) for s in path_st)
    print("{:<6} {:<20} {:<40} {:<10} {}".format(
        start, "ST (sideways<=10)", path_str, term_st, "YES" if stuck_st else "no"))
    print()

print("FC stuck (with sideways): {} / 12 (reached global max: {} / 12)".format(
    fc_stuck_sw, fc_global_sw))
print("ST stuck (with sideways): {} / 12 (reached global max: {} / 12)".format(
    st_stuck_sw, st_global_sw))
print()
print("=" * 75)
print("COMPARISON: Success rates before vs after sideways-move fix")
print("{:<22} {:<28} {}".format("Algorithm", "Before (no sideways)", "After (sideways<=10)"))
print("-" * 65)
print("{:<22} {} / 12 ({:.0f}%) {:<18} {} / 12 ({:.0f}%)".format(
    "First-Choice HC", fc_global_p, 100 * fc_global_p / 12, "", fc_global_sw, 100 * fc_global_sw / 12))
print("{:<22} {} / 12 ({:.0f}%) {:<18} {} / 12 ({:.0f}%)".format(
    "Stochastic HC", st_global_p, 100 * st_global_p / 12, "", st_global_sw, 100 * st_global_sw / 12))

if __name__ == "__main__":
    main()
    run_plateau_experiments()

```


=====

PART (a)/(b) -- Original Landscape

State f(s)

1	4
2	9
3	6
4	11
5	8
6	15
7	10
8	7
9	13
10	5
11	16
12	12

Start	Algorithm	Path	Terminal	Steps

1	First-Choice HC	1 -> 2	2	1
1	Stochastic HC	1 -> 2	2	1
2	First-Choice HC	2	2	0
2	Stochastic HC	2	2	0
3	First-Choice HC	3 -> 2	2	1
3	Stochastic HC	3 -> 2	2	1
4	First-Choice HC	4	4	0
4	Stochastic HC	4	4	0

5	First-Choice HC	5 -> 4	4	1
5	Stochastic HC	5 -> 6	6	1
6	First-Choice HC	6	6	0
6	Stochastic HC	6	6	0
7	First-Choice HC	7 -> 6	6	1
7	Stochastic HC	7 -> 6	6	1
8	First-Choice HC	8 -> 7 -> 6	6	2
8	Stochastic HC	8 -> 7 -> 6	6	2
9	First-Choice HC	9	9	0
9	Stochastic HC	9	9	0
10	First-Choice HC	10 -> 9	9	1
10	Stochastic HC	10 -> 9	9	1
11	First-Choice HC	11	11	0
11	Stochastic HC	11	11	0
12	First-Choice HC	12 -> 11	11	1
12	Stochastic HC	12 -> 11	11	1

```
=====
SUMMARY TABLE: Starting states reaching global maximum (state 11, f=16)
Algorithm          Count reaching state 11   Out of 12
-----
First-Choice HC      2                      12
```

```
Stochastic HC      2                      12
```

```
=====
DIVERGENCE: States where FC and Stochastic HC reach DIFFERENT terminals
Start  FC Terminal   ST Terminal   Reason
-----
5      4             6             FC chose left first (f=11); ST chose right (f=15)
```

```
=====
50-RUN EXPERIMENT: Stochastic HC from start = 4 (no fixed seed)
```

```
-----
Terminal State   Count   Percentage
-----
State 4          50      100.0% << local max
```

```
Reached global max (state 11): 0/50 = 0.0%
Stuck at local max:          50/50 = 100.0%
```

```
=====
PART (c) -- PLATEAU LANDSCAPE: f(5)=f(6)=f(7)=15
```

```
State  f(s)
1      4
2      9
3      6
4     11
5     15
6     15
7     15
8      7
```

RUN WITHOUT SIDEWAYS MOVES (plateau detection only)

Start	Algorithm	Path	Terminal	Stuck?
1	FC (no sideways)	1 -> 2	2	YES
1	ST (no sideways)	1 -> 2	2	YES
2	FC (no sideways)	2	2	YES
2	ST (no sideways)	2	2	YES
3	FC (no sideways)	3 -> 2	2	YES
3	ST (no sideways)	3 -> 2	2	YES
	[PLATEAU WARNING] FC: state 5 (f=15) -- no strictly better neighbour but equal neighbour exists.			
4	FC (no sideways)	4 -> 5	5	YES
	[PLATEAU WARNING] ST: state 5 (f=15) -- no strictly better neighbour but equal neighbour exists.			
4	ST (no sideways)	4 -> 5	5	YES
	[PLATEAU WARNING] FC: state 5 (f=15) -- no strictly better neighbour but equal neighbour exists.			
5	FC (no sideways)	5	5	YES
	[PLATEAU WARNING] ST: state 5 (f=15) -- no strictly better neighbour but equal neighbour exists.			
5	ST (no sideways)	5	5	YES
	[PLATEAU WARNING] FC: state 6 (f=15) -- no strictly better neighbour but equal neighbour exists.			
6	FC (no sideways)	6	6	YES
	[PLATEAU WARNING] ST: state 6 (f=15) -- no strictly better neighbour but equal neighbour exists.			
6	ST (no sideways)	6	6	YES
6	ST (no sideways)	6	6	YES
	[PLATEAU WARNING] FC: state 7 (f=15) -- no strictly better neighbour but equal neighbour exists.			
7	FC (no sideways)	7	7	YES
	[PLATEAU WARNING] ST: state 7 (f=15) -- no strictly better neighbour but equal neighbour exists.			
7	ST (no sideways)	7	7	YES
	[PLATEAU WARNING] FC: state 7 (f=15) -- no strictly better neighbour but equal neighbour exists.			
8	FC (no sideways)	8 -> 7	7	YES
	[PLATEAU WARNING] ST: state 7 (f=15) -- no strictly better neighbour but equal neighbour exists.			
8	ST (no sideways)	8 -> 7	7	YES
9	FC (no sideways)	9	9	YES
9	ST (no sideways)	9	9	YES
10	FC (no sideways)	10 -> 9	9	YES
10	ST (no sideways)	10 -> 9	9	YES
11	FC (no sideways)	11	11	no
11	ST (no sideways)	11	11	no
12	FC (no sideways)	12 -> 11	11	no
12	ST (no sideways)	12 -> 11	11	no
FC stuck on plateau: 10/12 (reached global max: 2/12)				
ST stuck on plateau: 10/12 (reached global max: 2/12)				

RUN WITH SIDEWAYS MOVES (cap = 10 per run)

[illegible]

[illegible]

```

[PLATEAU WARNING] ST: state 7 (f=15) -- no strictly better neighbour but equal neighbour exists.
[PLATEAU WARNING] ST: state 6 (f=15) -- no strictly better neighbour but equal neighbour exists.
[PLATEAU WARNING] ST: state 7 (f=15) -- no strictly better neighbour but equal neighbour exists.
8      ST (sideways<=10)    8 -> 7 -> 6 -> 5 -> 6 -> 5 -> 6 -> 5 -> 6 -> 7 -> 6 -> 7 7      YES

9      FC (sideways<=10)    9                                9      YES
9      ST (sideways<=10)    9                                9      YES

10     FC (sideways<=10)    10 -> 9                            9      YES
10     ST (sideways<=10)    10 -> 9                            9      YES

11     FC (sideways<=10)    11                                11     no
11     ST (sideways<=10)    11                                11     no

12     FC (sideways<=10)    12 -> 11                          11     no
12     ST (sideways<=10)    12 -> 11                          11     no

FC stuck (with sideways): 10/12 (reached global max: 2/12)
ST stuck (with sideways): 10/12 (reached global max: 2/12)

=====
COMPARISON: Success rates before vs after sideways-move fix
Algorithm          Before (no sideways)      After (sideways<=10)
-----
First-Choice HC    2/12 (17%)                2/12 (17%)
Stochastic HC      2/12 (17%)                2/12 (17%)

```

Discussion:

The sideways-move fix does not improve success rates for either First-Choice or Stochastic Hill Climbing in this plateau landscape — both variants still reach the global maximum only 2 out of 12 times. Both algorithms benefit equally in terms of being able to explore the plateau without immediately stopping, but this does not help reach the global maximum because the plateau blocks any strictly uphill path. The First-Choice HC tends to oscillate between adjacent equal neighbors (e.g., $5 \leftrightarrow 6$), while Stochastic HC sometimes jumps among all equal neighbors randomly ($5 \leftrightarrow 6 \leftrightarrow 7$), so sideways moves allow more exploration but still can't escape the plateau. The cap of 10 sideways moves is important because it prevents infinite cycling between equal-valued states, ensuring the algorithm eventually stops rather than looping endlessly. Overall, sideways moves help the algorithm handle plateaus without crashing, but they do not guarantee finding the global optimum.